CompSci 130
Introduction to Software Fundamentals

Software Maintenance, Modularity, Exceptions & Testing

Learning outcomes

- At the end of this lecture, you should be able to:
  - Describe how software failure can have significant effects
  - Use test cases as part of program development
  - Understand the flow of control that occurs with exceptions
    - try, except, finally
  - Use exceptions to handle unexpected runtime errors gracefully
    - 'catching' an exception of the appropriate type
  - Generate exceptions when appropriate
    - raise an exception
- Further resources on errors and exceptions:
  - Online Textbook: https://onlinetextbook.auckland.ac.nz/02-modular-design/
  - https://docs.python.org/3/tutorial/errors.html
  - http://www.python-course.eu/python3_exception_handling.php

Modular Design

- Programs usually consist of **several** parts that interact with each other
  - each part is relatively independent and self-contained
  - the code will be easier to understand and easier to modify.
- Good software is also designed to be **modular**.
  - replace individual parts of the software system when the system needs to be modified, without rewriting the entire program.
  - Each component is
    - relatively short and
    - easy to understand
  - If a component needs to be modified, the changes to that component can be tested **without** having to run the entire program each time.

Realities of software development

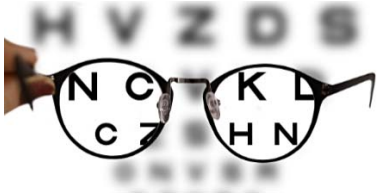
- Most of the lifetime **cost** of a software system occurs after its initial development
- Most software systems have **multiple** developers, and few developers work just on code they have written
- Successful software systems live for **many years**
- Much of the lifetime cost of a software system comes from **changing** existing code
  - Part of the cost of changing existing code is the time required to understand it
  - But some choices of code structure are harder to change than others
- Anything that makes it hard to change code **increases** the cost of software development
- Improving maintainability **reduces** the cost of software
  - making code easier to **understand**
  - making code easier to **change**

Software maintainability

- **Maintainability**
  - Effort required to keep the software useful.
  - **Perfective** maintenance
    - adding new features (typically 50% of the effort or more)
  - **Corrective** maintenance
    - fixing faults ("bugs") (typically less than 20% of the effort)
  - **Preventative** maintenance
    - improving some aspect of the software in anticipation of potential problems in the future (by "refactoring") (typically less than 5% of the effort)
  - **Adaptive** maintenance
    - moving the same functionality to a new environment (e.g. new hardware, new operating system)
    - adjusting to changes in environment (e.g. changes in legislation)

Characteristics of maintainable code

- Written in a good and consistent style
  - Poor style leads to higher costs in software development as
    - difficult to read
    - difficult to understand and change in future



- Well documented:
  - need to understand what the code does
    - use descriptive variable and function names.
    - use docstrings
    - use comments in code
- Modular.
  - Modular code is code that is divided into functions. Each function should perform **a specific and clearly** defined task.
    - These functions should be independent of each other
      - changes made to one function don't result in having to make changes to other functions

How can we avoid mistakes?

- You can't!

- Write readable code
  - Make your intention explicit
- Document your code
  - But do it only when absolutely necessary (readable code is the best documentation!)

- Test your code thoroughly
  - Check if it matches specification/documentation
  - Does it do what you claim?

Expensive Fireworks (1996)

- In 1996, code from the Ariane 4 rocket was reused in the Ariane 5, but the new rocket's faster engines trigger a bug in an arithmetic routine inside the flight computer.
- The error was in code to convert 64-bit floating-point numbers to 16-bit signed integers. The faster engines caused the 64-bit numbers to be larger, triggering an overflow condition that crashed the flight computer.
- As a result, the rocket's primary processor overpowered the rocket's engines and caused the rocket to disintegrate only 40 seconds after launch.

How do you know if your code works?

- Syntax errors
  - Instructions cannot be understood by the compiler/interpreter
  - Compiler / Interpreter tells you what it doesn't unders
    - Understanding error messages is critically important

IndexError: tuple index out of range

```
my_string = "{}, I am {} years old and my {} is {}"
print(my_string.format("Hello", "18", "Mike"))
```

- Runtime errors
  - Program compiles and can be executed
  - Error cause a program to not execute properly e.g. a division by zero

```
>>> 4 / 0
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in
<module>
    4 / 0
ZeroDivisionError: division by zero
>>>
```

- Logical errors
  - Doesn't produce the desired result
  - These generally because you didn't consider a certain execution scenario

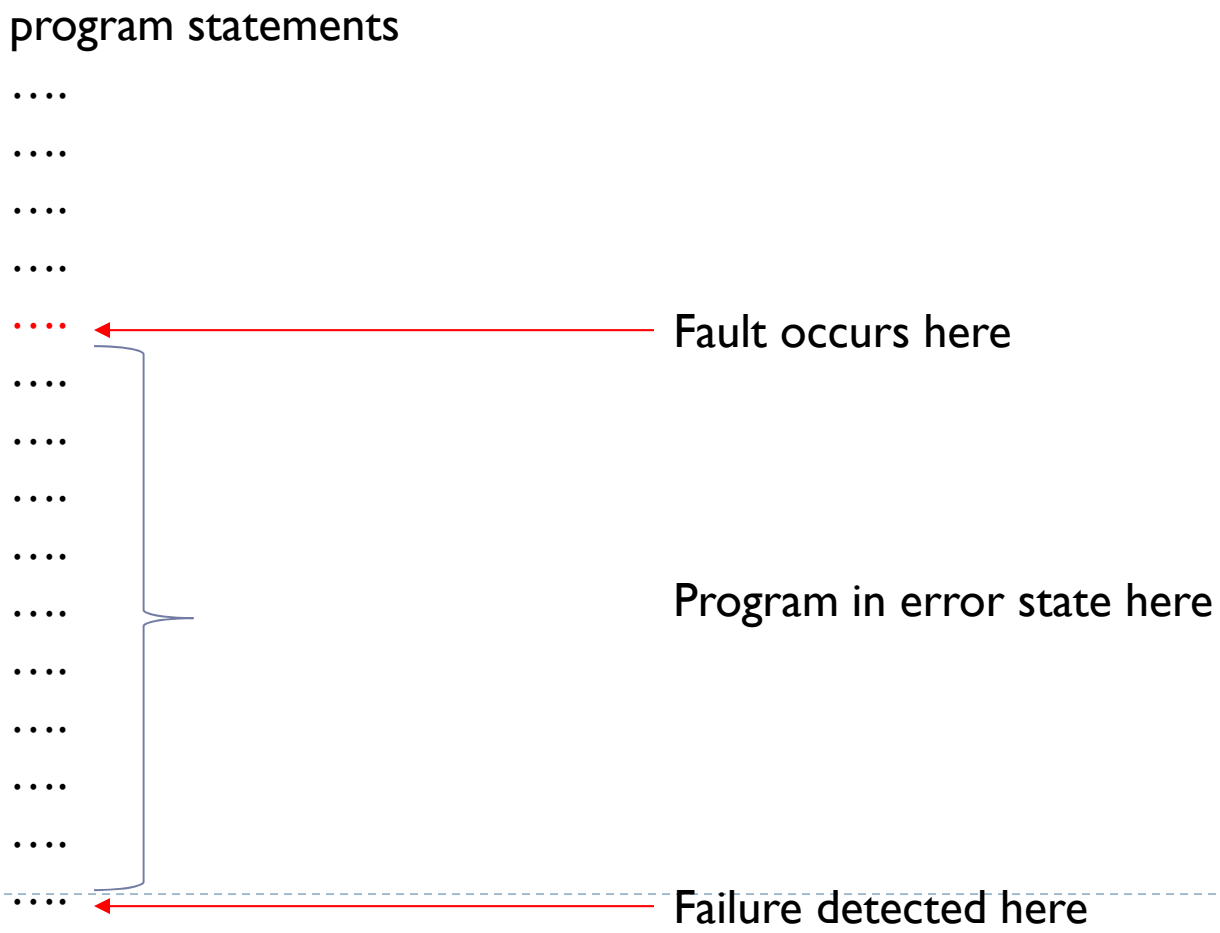
```
def compute_power_of_two(number):
   return number * 2
```

Error Terminology

- Error
  - Human mistake, misconception, misunderstanding
- Fault
  - Part of the program that causes it to behave incorrectly.
  - Often called a bug, or defect.
  - Causes the internal state of the program (contents of the memory, or execution path) to be incorrect
- Failure
  - The observed behaviour of a program differs from the expected/required behaviour

Faults and failures

- Failures are not usually observed at the point of the fault
  - Faults do not always cause program to be in an error state



Diagnosing a problem

- Debugging is the act of diagnosis
  - Start with the symptoms
    - Conduct more tests to collect more data
  - Develop a hypothesis
    - Understanding the control flow of a program is critical
      - Tracing the changes to information is a useful tool
  - Test the hypothesis
- Program Testing
  - Testing is the process of determining whether software behaves the way we require it to behave.
  - Approaches:
    - Walking through your code
    - Giving your code to someone else and asking them to test it
    - Using testing software to automate the testing process

Detecting faults

- Before you write the code, figure out what output you expect
  - **A test case** is a combination of the **input** to a program and the **expected output** or behaviour of the program
  - Extreme case: write all test cases before you write the actual code logic (test driven development)!!!

$$area\ of\ a\ triangle = \frac{height}{2} \times base$$

  - Example: Write a function that takes two integers as parameters and calculates the area of a triangle

Height	Base	Area
20	123	1230
2	2	2

  - **Do I have enough test cases?**
    - Are 2 test cases enough to determine that a function is functioning correctly and if not?
    - how do we know we have defined enough test cases?

Exercise 1: $area\ of\ a\ triangle = \frac{height}{2} \times base$

CodeRunner

- Write a list of test cases that would usefully test a function that calculates the area of a triangle.
  - Compare your list with test cases generated by your peers. Are some of the cases redundant? What is each case testing?
  - Have you defined enough test cases to know your code is working perfectly?
  - It is impossible to test every possible case
  - Therefore, the next best thing is to provide some guidelines that will minimize the chances of missing important test cases.

Guidelines

- Consider Edge Cases
  - At least one test case for each value that is treated differently
  - For the triangle area example, how about 0 or negative values?
- Consider Unexpected Values
  - For the triangle area example, what happens if a string is supplied?
- Consider Different Data Types
  - For the triangle area example, is it ok to enter by floating point values
- Consider Conditional Statements
  - Are there conditional statements in your code that result in different branches of your code being executed?
    - at least one test case the executes each branch of the code
- Redundant Test Case
  - A redundant test case is one, which if removed, will not affect the fault detection effectiveness of the testing process

doctest

- doctest is a simple system for testing code
  - Tests are included in the docstring for a function
  - Simple way to perform unit testing (Python has more advanced ways)
  - Any line that begins with the Python prompt >>> will be executed
  - The output from executing the code will be compared with the line that follows

```
>>> get_triangle_area(10, 2)
10.0
```

include this inside your docstring for the calculate_average function

- To test the code, include the following statements in the module

```
import doctest
doctest.testmod()
```

Example

```
def get_triangle_area(height, base):
    """Returns the area of a triangle.
    >>> get_triangle_area(10.5, 2)
    10.5
    >>> get_triangle_area(0, 10)
    0.0
    >>> get_triangle_area(20, 0)
    0.0
    >>> get_triangle_area(-1, 20)
    -10.0
    >>> get_triangle_area(10, -1)
    -5.0
    """
    return base * height / 2
```

Program Development

- Work out exactly **what** the program needs to do
  - Think of different input
  - Manually work out what output would be produced
  - Write the input and output in a table of test cases
- Work on the solution, keeping the **test cases** in mind
  - Manually work out the solution by hand to each test case
  - Write down how you solved the problem
  - Generalise the solution to an algorithm
- Convert the **algorithm** into programming code
  - Test your code after each step
- When you are finished
  - **Check** your program against your original test cases
  - Add more tests and check as thoroughly as you can
  - Especially consider "**edge cases**" and **unexpected input**

Testing Strategies

- Offensive programming
  - Force the program to fail immediately if something goes wrong

- Assert
  - Specifies a condition that must be True
  - Terminates the program immediately if condition is False

assert <condition> [, <message>]

boolean

optional string

```
even_num = 3
assert even_num % 2 == 0,“even_num should be even but {} is odd”.format(even_num)
```

Debugging

- Debugging requires you to understand the flow of control and how data is stored.

- To debug your code, you need to know:
  - what your code *should* produce
  - what your code *does* produce

- Try to identify when the program went into an error state
  - Using print statements to check contents of variables
  - Stepping through the code one line at a time using a debugger software

Handling of errors

- Errors are inevitable in software programs. However, if you handle errors properly, you'll greatly improve your program's readability, reliability and maintainability.

- Often, we are aware of errors that may happen during execution of a program, but we do not want to terminate a program just because an error happened!

- We may want to break the normal flow of code, but still be able to continue the normal flow, if the error has been dealt with. This is also called **defensive programming**.

Example01.py

Handling unexpected input values

- Let's think about: What if this function is passed a value that causes a divide by zero?
  - Error caused at runtime
  - Error occurs within the function
  - Problem is with the input

```
def divide(numerator, denominator):
    return numerator / denominator

value = divide(5, 0)
print(value)
```

- What can we do?
  - You can try to check for valid input first

where

```
Traceback (most recent call last):
  File "...", line 4, in <module> x = divide(5, 0)
  File "...", line 2, in divide  return numerator / denominator
ZeroDivisionError: division by zero
```

reason

Divide by zero error

- One could: check for valid input first…
  - Only accept input where the denominator is **non-zero**

```
def divide(numerator, denominator):
    if denominator == 0:
        print('Error: cannot divide by zero')
    else:
        return numerator / denominator
```

  - But: what if "denominator" is not a number?

```
def divide(numerator, denominator):
    if (type(denominator) is not int and
        type(denominator) is not float):
            print("Error: denominator is not a number")
    elif denominator == 0:
            print('Error: cannot divide by zero‘)
...
```

Handling input error

- Check for valid input first
  - What if "numerator" is not a number?

That's too long!

```
def divide(numerator, denominator):
    if (type(denominator) is not int and
        type(denominator) is not float or
        type(numerator) is not int and
        type(numerator) is not float):
            print('Error: one or more operands' +
                          ' is not a number')
    elif denominator == 0:
         print('Error: cannot divide by zero')
    else:
        return numerator / denominator

print(divide(5, 'hello'))
```

Exceptions

- We can use exceptions to handle errors that break the flow of code
  - Python provides exceptions for error handling
- Exception examples:
  - Attempt to divide by zero
  - Couldn't find the specific file to read

ZeroDivisionError
IndexError
TypeError
ValueError
AttributeError
FileNotFoundError

- If an exception occurs, the run-time system will attempt to handle the exception (default exception handler), usually by displaying an error message and terminating the program.

What is an Exception?

- An exception is an event that occurs during the execution of a program that **disrupts** the normal flow of instructions during the execution of a program.
- When an error occurs within a function, the function creates an exception object and hands it over to the runtime system.
- The exception object contains
  - information about the error, including its type and the state of the program when the error occurred.
- Creating an exception object and handing it to the runtime system is called throwing an exception.

Handling exceptions

- Code that might create a runtime error is enclosed in a try block
  - Statements are executed sequentially as normal
  - If an error occurs then the remainder of the code is skipped
  - The code starts executing again at the except clause
    - The exception is "caught"

```
try:
    statement block
    statement block
except:
    exception handling statements
    exception handling statements
```

- Advantages of catching exceptions:
  - It allows you to fix the error in a dedicated code segment
  - It prevents the program from automatically terminating

Case 1

Example02.py

```
def divide(numerator, denominator):
    try:
        result = numerator / denominator
        print('calculating ...')
        return result
    except:
        print('Error in input data')
```

- Case 1: No error
  - divide(5,5)

2 ✓
3 ✓

```
print(divide(5, 1))
print("Program can continue to run...")
```

```
calculating ...
5.0
Program can continue to run...
```

Case 2

```
def divide(numerator, denominator):
    try:
        result = numerator / denominator
        print('calculating ...')
        return result
    except:
        print('Error in input data')
```

- Case 2: Invalid input
  - 1. call divide(5,0)
  - 2. call divide(5, 'Hello')

```
print(divide(5, 0))
print("Program can continue to run...")
```

```
Error in input data
None
Program can continue to run...
```

  - But what is the error in each situation?
    - 1) 5/0 => ZeroDivisionError: division by zero
    - 2) 5/'hello' =>TypeError: unsupported operand type(s) for /: 'int' and 'str'

Danger in catching all exceptions

- The general **except** clause catches **all** runtime errors
  - Sometimes that can hide problems
  - IOError or IndexError if file empty? Caller has no clue!
- You can **put two or more** except clauses, each except block is an exception handler and handles the type of exception indicated by its argument in a program.
  - The runtime system invokes the exception handler when the handler is the **FIRST ONE** that matches the **type** of the exception thrown.
    - It executes the statement inside the matched except block, the other except blocks are bypassed and continue after the try-except block.

Specifying the exceptions

Example03.py

```
def divide(numerator, denominator):
    try:
        return numerator / denominator
    except TypeError:
        print('Type of operands is incorrect')
    except ZeroDivisionError:
        print('Divided by zero')
```

- Case 1:
  - No error

```
print(divide(5, 5))
```

```
1.0
```

Specifying the exceptions

Example03.py

```
def divide(numerator, denominator):
    try:
        return numerator / denominator
    except TypeError:
        print('Type of operands is incorrect')
    except ZeroDivisionError:
        print('Divided by zero')
```

- Case 2:
  - is not a number

```
print(divide('abc', 0))
```

2

```
Type of operands is incorrect
None
```

Specifying the exceptions

Example03.py

```
def divide(numerator, denominator):
    try:
        return numerator / denominator
    except TypeError:
        print('Type of operands is incorrect')
    except ZeroDivisionError:
        print('Divided by zero')
```

- Case 3:
  - Division Error

```
print(divide(5, 0))
```

2

```
Divided by zero
None
```

Example04.py

Exception not matched

- If no matching except block is found, the run-time system will attempt to handle the exception, by terminating the program.

```
def divide(numerator, denominator):
    try:
        return numerator / denominator
    except ValueError:
        print('Invalid value')
    except ZeroDivisionError:
        print('Divided by zero')
```

```
print(divide('abc', 0))
```

```
Traceback (most recent call last):
  File "...", line 11, in <module> print ...
  File "...", line 3, in divide
    return numerator / denominator
unsupported operand type(s) for /: 'str' and 'int'
```

Example05.py

Order of except clauses

- Specific exception blocks must come before a general exception block (but use the general ones sparingly!)

```
def divide(numerator, denominator):
    try:
        return numerator / denominator
    except:
        print('Type of operands is incorrect')
    except ZeroDivisionError:
        print('Divided by zero')
```

```
return numerator / denominator
SyntaxError: default 'except:' must be last
```

```
try:
  ...
except ZeroDivisionError:
  ...
except:
  ...
```

Exceptions

- Any kind of built-in error can be caught
  - Check the Python documentation for the complete list
  - Some common errors:
    - ArithmeticError: various arithmetic errors
    - ZeroDivisionError
    - IndexError: a sequence subscript is out of range
    - TypeError: inappropriate type
    - ValueError:
      - has the right type but an inappropriate value
    - IOError: Raised when an I/O operation fails
    - FileNotFoundError
    - EOFError:
      - encounter an end-of-file condition (EOF) without reading any data

```
BaseException
 +-- SystemExit
 +-- KeyboardInterrupt
 +-- GeneratorExit
 +-- Exception
      +-- StopIteration
      +-- ArithmeticError
      |    +-- FloatingPointError
      |    +-- OverflowError
      |    +-- ZeroDivisionError
      +-- AssertionError
      +-- AttributeError
      +-- BufferError
      +-- EOFError
      +-- ImportError
      +-- LookupError
      |    +-- IndexError
      |    +-- KeyError
      +-- MemoryError
      +-- NameError
      |    +-- UnboundLocalError
      +-- OSError
      |    +-- BlockingIOError
      |    +-- ChildProcessError
      |    +-- ConnectionError
      |    |    +-- BrokenPipeError
      |    |    +-- ConnectionAbortedError
      |    |    +-- ConnectionRefusedError
      |    |    +-- ConnectionResetError
      |    +-- FileExistsError
      |    +-- FileNotFoundError
      |    +-- InterruptedError
      |    +-- IsADirectoryError
      |    +-- NotADirectoryError
      |    +-- PermissionError
      |    +-- ProcessLookupError
      |    +-- TimeoutError
      +-- ReferenceError
      +-- RuntimeError
      |    +-- NotImplementedError
      +-- SyntaxError
      |    +-- IndentationError
      |         +-- TabError
      +-- SystemError
      +-- TypeError
      +-- ValueError
      |    +-- UnicodeError
      |         +-- UnicodeDecodeError
      |         +-- UnicodeEncodeError
      |         +-- UnicodeTranslateError
```

TypeError vs. ValueError

- TypeErrors are caused by combining the wrong type of objects or calling a function with the wrong type of object.
  - This happens when someone tries to do an operation with different kinds of incompatible data types. A common example is to do addition of an integer and a string.
  - print (2 + "a")

- A ValueError is used when a function receives a value that has the right type but an invalid value
  - value = int('a')
  - value = float('a')

Exercise 03

- Consider the code below:

```
my_list = [1, 2, 3]
index = int(input('Enter an index: '))
print(my_list[index])
```

Enter an index: 6
...
IndexError: list index out of range

Enter an index: 1
2

Enter an index: 6
Invalid index!

- Rewrite it using a try…except block to handle the IndexError

```
try:
  my_list = [1, 2, 3]
```

CodeRunner

Exercise 04

- Consider the code below:

```
my_dict = {'test1':1,'test2':2}
key = input('Enter a key: ')
print(my_dict[key])
```

Enter a key: unknown
...
KeyError: 'unknown'

Enter a key: test1
1

Enter a key: test
Invalid Key!

- Rewrite it using a try…except block to handle the KeyError

```
try:
  my_dict = {'test1':1,'test2':2}
```

The else clause

- Executed only if the try clause completes with no errors
  - It is useful for code that must be executed if the try clause does not raise an exception.

```
try:
    statement block here
except:
    more statements here (undo operations)
else:
    more statements here (close operations)
```

Examples

Example07.py

```
try:
    age = int(input("Please enter your age: "))
except ValueError:
    print("Hey, that wasn't a number!")
else:
    print("I see that you are {} years old.".format(age))
```

```
Please enter your age: 4
I see that you are 4 years old.
```

```
Please enter your age: a
Hey, that wasn't a number!
```

Exercise 5a, 5b

- What is the output of the following code fragment?

```
try:
    my_list = [1, 2, 3]
    num = int(input('Enter an index: '))
    value = my_list[num]
except IndexError:
    print("Invalid index!")
else:
    print(value)
print("DONE")
```

  - Cases:
    - Enter an index: 1
    - Enter an index: 6

The finally clause

- The finally block is optional, and is not frequently used
- Executed after the try and except blocks, but before the entire try-except ends
- Code within a finally block is **guaranteed** to be executed if any part of the associated try block is executed regardless of an exception being thrown or not
  - It allows for cleanup of actions that occurred in the try block but may remain undone if an exception is caught
  - Often used with files to close the file

```
try:
    statement block here
except:
    more statements here (undo operations)
finally:
    more statements here
```

Example

Example08.py

```
def divide(numerator, denominator):
    try:
        result = numerator / denominator
    except ZeroDivisionError:
        print('Divided by zero')
        result = 'N/A'
    else:
        print("Result is", result)
    finally:
        print("finally clause")
    return result
```

- Case 1:
  - No error

```
print(divide(2, 1))
```

```
Result is 2.0
finally clause
2.0
```

Example

Example08.py

```
def divide(numerator, denominator):
    try:
        result = numerator / denominator
    except ZeroDivisionError:
        print('Divided by zero')
        result = 'N/A'
    else:
        print("Result is", result)
    finally:
        print("finally clause")
    return result
```

- Case 2:
  - Divided by zero

```
print(divide(2, 0))
```

```
Divided by zero
finally clause
N/A
```

Example

Example08.py

```
def divide(numerator, denominator):
 try:
        result = numerator / denominator
    except ZeroDivisionError:
        print('Divided by zero')
        result = 'N/A'
    else:
        print("Result is", result)
    finally:
        print("finally clause")
    return result
```

- Case 3:
  - Other error

```
print(divide('2', '1'))
```

```
finally clause
Traceback (most ...
TypeError: unsupported operand type(s) ...
```

Exercise 06a, 6b

- What is the output of the following code fragment?

```
try:
    age = int(input("Please enter your age: "))
except ValueError:
    print("Hey, that wasn't a number!")
else:
    print("I see that you are %d years old." % age)
finally:
    print("It was really nice talking to you.  Goodbye!")
```

  - Cases:
    - Please enter your age: a
    - Please enter your age: -1
    - Please enter your age: 4

Exception Handling for FileIO

- What to do when a file can not be found (FileNotFoundError) or there is a problem opening the file
- In general

```
try:
      Attempt something where exception error may happen
      (i.e. open a file and read the content)

except FileNotFoundError:
      React to the error

else:
      What to do if no error is encountered
     (i.e. close the file)

finally:
      Actions that must always be performed
```

Exceptions: File Example

- Consider the following code:

```
try:
    filename = input("Enter name of input file: ")
    input_file = open (filename, "r")
    one_line = input_file.readline()
except FileNotFoundError:
    print("File", filename, "could not be opened")
else:
    print(one_line)
    input_file.close()
    print ("Closed file", filename)
```

```
Enter name of input file: numbers.txt
43 34

Closed file numbers.txt
```

- Case 1:
- Case 2:

```
Enter name of input file: test.txt
File test.txt could not be opened
```

Raising an exception:

- You can create an exception by using the raise statement

```
raise Error('Error message goes here')
```

  - The program stops immediately after the raise statement; and any subsequent statements are not executed.
  - It is normally used for testing and debugging purpose

- Example:

```
def check_age(age):
    if age < 0:
        raise ValueError('Invalid age!')
    else:
        print(age)

check_age(0)
```

```
Traceback (most recent call last):
  ...
  raise ValueError('Invalid age!')
ValueError: Invalid age!
```

CodeRunner

Exercise 07a

- Modify the following function:
  - returns the mean value of a list of numbers (if numbers are all valid)
  - returns an informative message when input is unexpected

```
def calculate_mean(data):
    total = 0
    for element in data:
       total += element
    mean =  total / len(data)
    return mean
```

```
print(mean([1,3,2,4,5]))
print(mean([1,3,'5',4,5]))
print(mean([1,3,'True',4,5]))
```

```
3.0
Error: Invalid number found!
Error: Invalid number found!
```

CodeRunner

Exercise 07b – Better Approach

- Modify the following function
  - returns the mean value of a list of numbers (even the list contains any invalid values)

```
def mean(data):
    total = 0
    for element in data:
       total += element
    mean =  total / len(data)
    return mean
```

```
print(mean([1,3,'5',4,5]))
print(mean([1,'True',3,4,5]))
```

```
3.25
3.25
```

Summary

- Decisions we make when developing software (intentional and unintentional) can have a significant impact on society (Ariane!).
- Testing is an essential part of software development
  - Tests should be written before the code
  - Writing tests helps you figure out what the code should do
  - Doctest is a simple system to automate the testing of code, python unittest and pytest modules are more sophisticated
- Exceptions alter the flow of control
  - When an exception is raised, execution stops
  - When the exception is caught, execution starts again
- try… except blocks are used to handle problem code
  - Make sure code fails gracefully
- finally executes necessary code after the exception handling